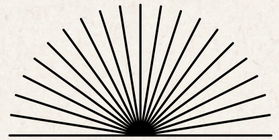
VEHICLE SIGNATURE DETECTION USING SURVEILLANCE AUDIO

Manavpreet Cheema, Nandini Goyal, Ona Dubey

MENTOR:
Professor Srikant Srinivasan



Flow

1	• Problem Statement • Proposed Solution • Gaps and why us
2	Objectives and how they were achieved
3	Progress: • Domain Concept, Insights • Research or Experimental data • Familiarity with coding packages
4	Pipeline and results
5	Literature review report

Problem Statement

This project aims to **use vehicle audio signatures for vehicle classification,** with the ultimate objective of enabling **reliable vehicle detection for surveillance applications,** especially in scenarios where visual data is unreliable. Specifically in the context of Plaksha.

State of the Art:

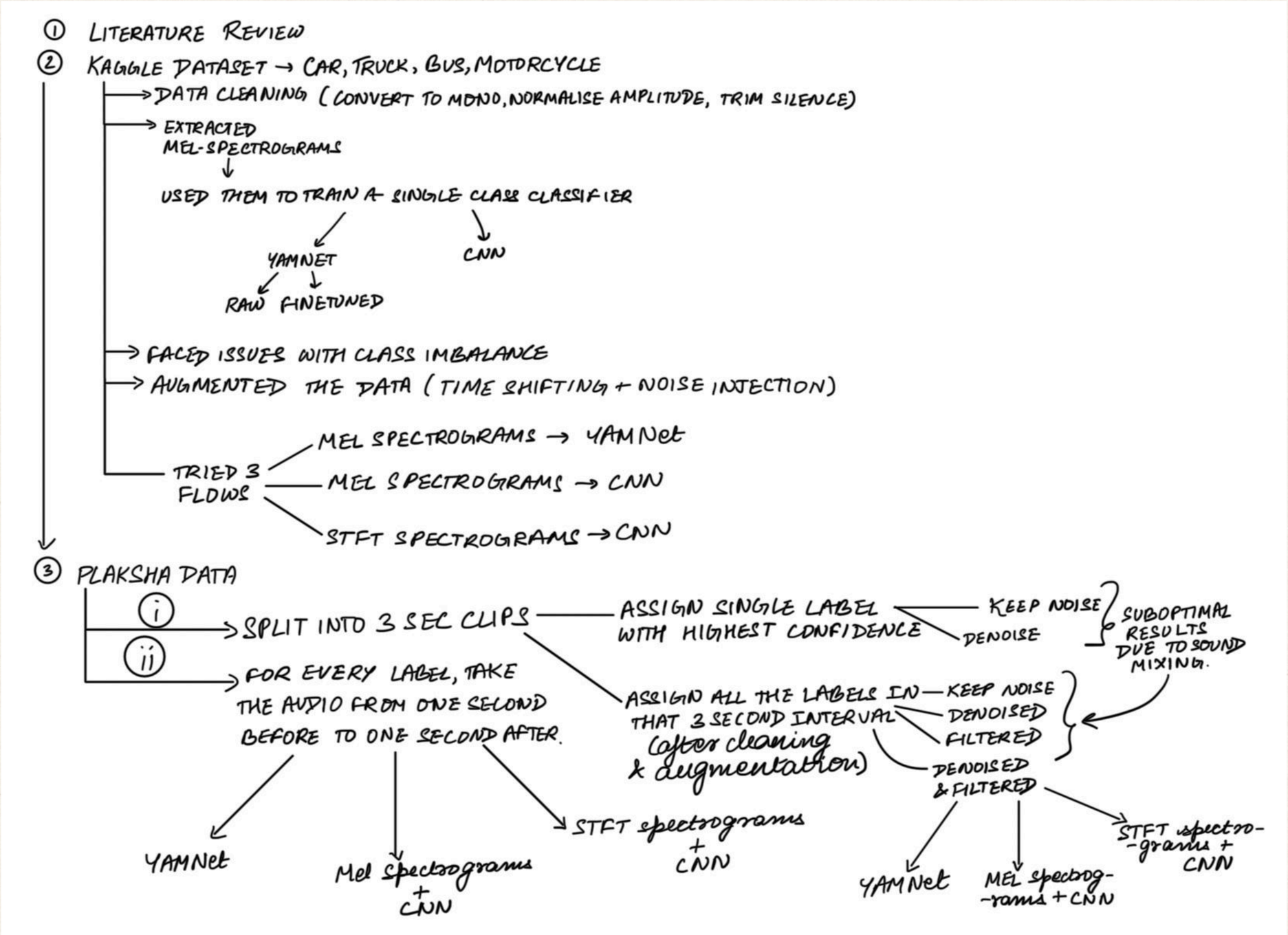
- **Acoustic-Based Detection:** A reliable alternative to vision-based methods, especially in low-visibility conditions.
  - **George et al. (2013):** Used ANN & k-NN with MFCCs for vehicle classification, addressing noise and overlapping sounds.
  - **GitHub Project:** Open-source ML-based sound detection for practical implementation.
- **Impact of our project:** High accuracy, real-world applicability, a proposed alternative to existing Computer Vision based methods.

Gaps	How we addressed them
Environmental Noise Sensitivity: Background noise and overlapping sounds still pose challenges despite preprocessing techniques.	We tried filtering and denoising the data: **Filter -** Using the bandpass filter of 5th order **Denoise -** Subtracting the dominant frequency of "no_vehicle" audio from the rest of the classes
Limited Generalisability : Existing models are usually trained on specific datasets, reducing their effectiveness in diverse traffic conditions.	We trained on 2 different datasets with 2 different models: 1. **Kaggle Dataset** - Took 4 classes from the dataset (Car, Truck, Bus, Motorcycle) and trained and tested on them with 2 models. 2. **Plaksha Dataset** - Used data and labels from the IoT lab to train our models on 4/5 classes (Car, Truck, Motorcycle, Autorickshaw, No_Vehicle)

OBJECTIVES

1. Develop a vehicle classification system using audio signatures for when visual data isn't reliable or available (for instance CV based classifiers cannot be used at night in some cases).
2. Tackle background noise issues by using filtering and denoising techniques to improve classification accuracy.
3. Train models on diverse data to make sure they work well in different traffic environments and aren’t limited to just one type of setting.
4. Work towards a system that can eventually run in real time on actual roads.
5. Integrate with Computer Vision/video based vehicle classifiers and detectors.

Progress - Overall flow



Progress - Domain Concept

Our CNN

Input:

- Grayscale spectrogram images of size 128×128
- Single channel input (1×128×128)

Classification Target
Multi-label prediction over 4 classes:
['auto', 'car', 'motorcycle', 'truck']

Output: Probabilities for each class (0 to 1), thresholded at 0.5

Why our Pipeline?

- **Audio is converted to Spectrogram for Input** - Takes in grayscale time-frequency images, CNN learns spatial patterns from temporal audio cues
- **Compact CNN Stack**: 4 conv layers + 2 fully connected block
- **Multi-Label Output (Sigmoid):** Supports overlapping vehicle sounds (e.g., car + truck), unlike softmax-based single-label classifiers.
- **Light - It's lighter than YAMNet, ResNet, DensNet, etc.** Small model ensures real world applicability, like on low-power devices for real-time deployment.
- **Dropout and pooling** - regularisation improves generalisation
- **Performed better than finetuned YAMNet** - a widely used and acknowledged pretrained model for vehicle sound recognition.
- **Made for Plaksha** - Initially worked with a Kaggle dataset (to solidify the flow), later switched to a dataset we made using data from the IoT Lab.

Convolutional Layers (self.conv_layers)

- Conv2d(1 → 16, kernel_size=3, padding=1)
  - ReLU()
  - MaxPool2d(kernel_size=2)
  - Output size: 16×64×64
- Conv2d(16 → 32, kernel_size=3, padding=1)
  - ReLU()
  - MaxPool2d(kernel_size=2)
  - Output size: 32×32×32
- Conv2d(32 → 64, kernel_size=3, padding=1)
  - ReLU()
  - MaxPool2d(kernel_size=2)
  - Output size: 64×16×16
- Conv2d(64 → 128, kernel_size=3, padding=1)
  - ReLU()
  - MaxPool2d(kernel_size=2)
  - Output size: 128×8×8

Flatten
Flattened tensor size: 128 × 8 × 8 = 8,192

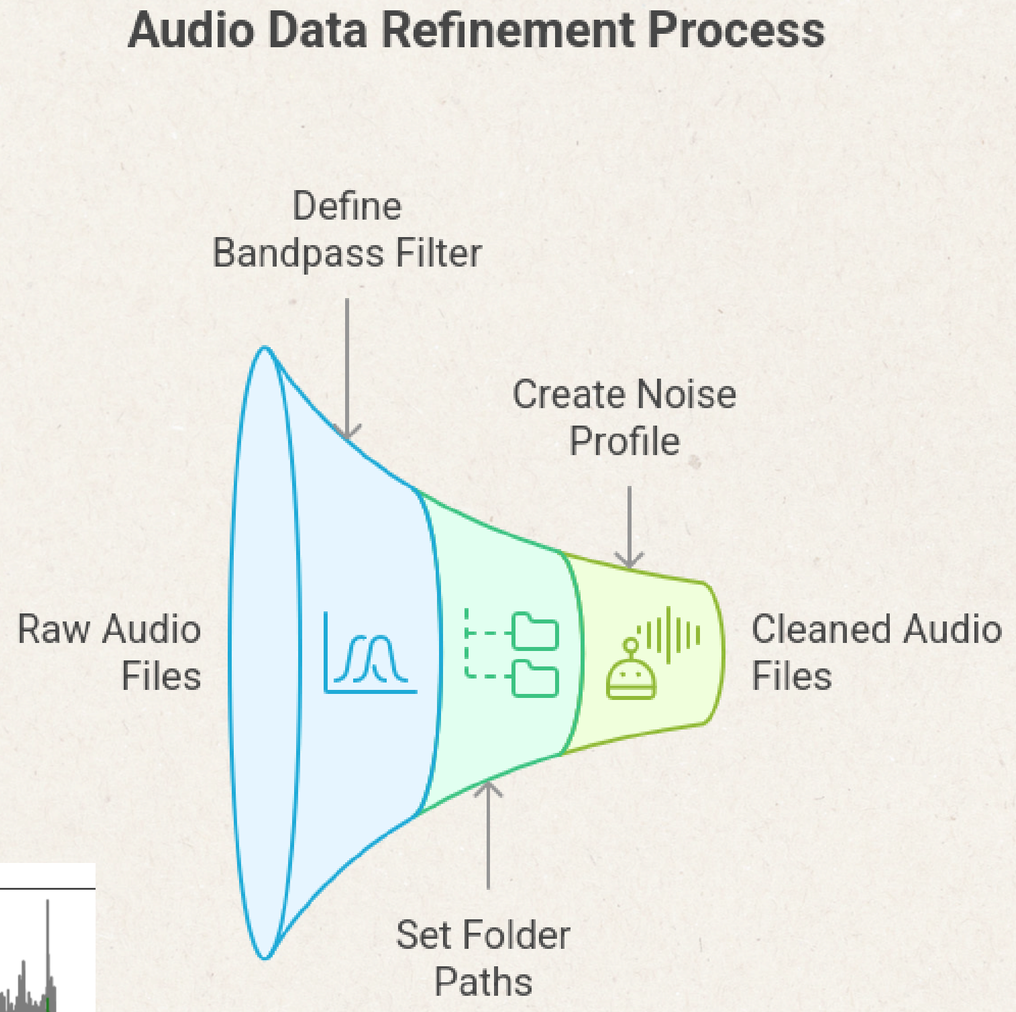
Fully Connected Layers (self.fc_layers)

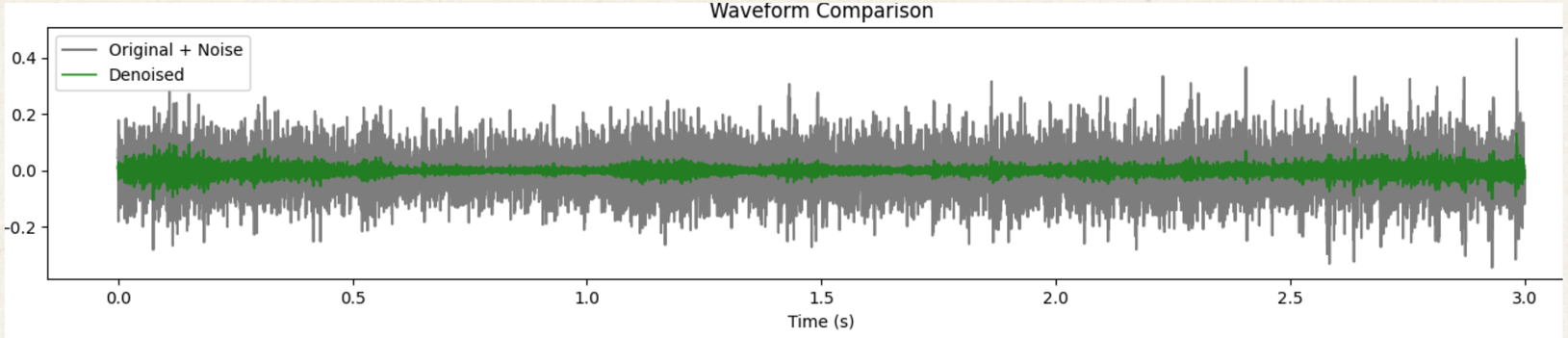
- Linear(8,192 → 256)
  - ReLU()
  - Dropout(p=0.4)
- Linear(256 → 128)
  - ReLU()
  - Dropout(p=0.3)
- Linear(128 → 4) (one output per class: auto, car, motorcycle, truck)
- Sigmoid() activation for multi-label classification

Progress - Domain Concept

NOISE REMOVAL

- For each audio file, compute its short-time Fourier transform (STFT) to get the frequency spectrum.
- Extract and store the average power spectrum (frequency energy) for each file.
- Calculate and print the dominant frequency range covering about 90% of the total energy
- Apply noise reduction using the averaged background noise.
- Save the cleaned audio to the corresponding output folder.



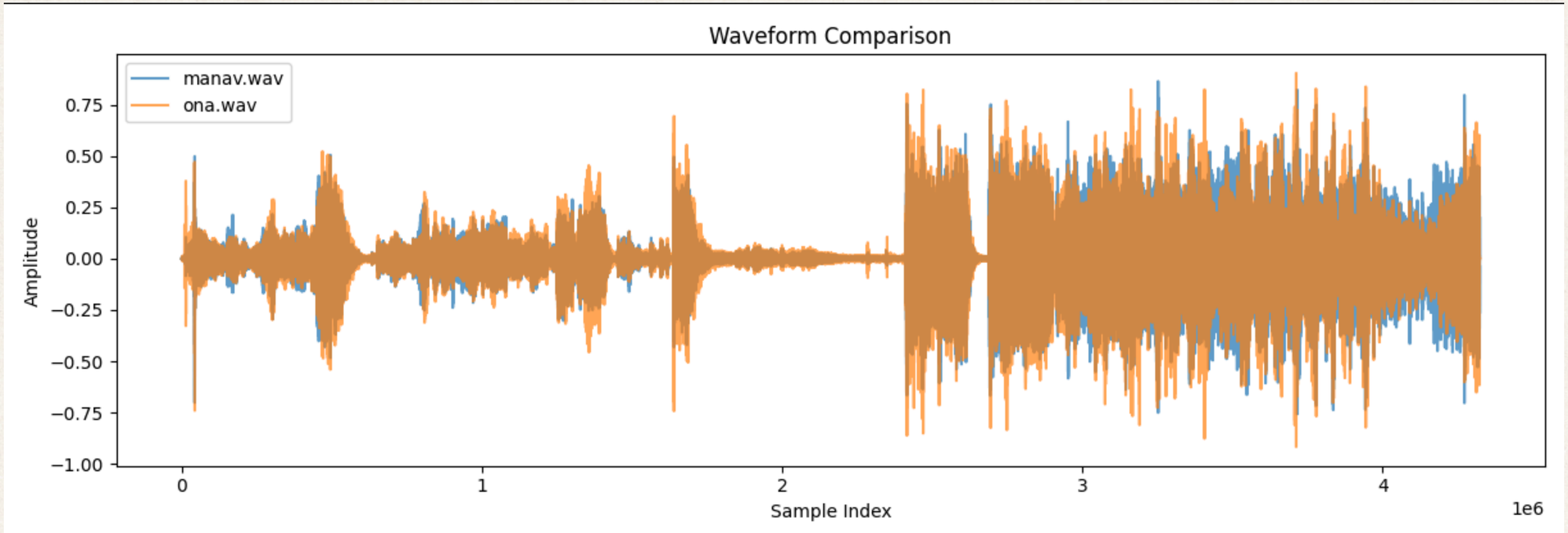


Progress - Domain Concept

MIC TESTING

The audio files (same audio)

- manav.wav - recorded from samsung device
- Ona.wav - recorded from apple device



Parameters (quality test parameters)

- **RMSE:** How close the waveforms are.
- **Spectrogram MSE:** Frequency-level difference.
- **PESQ**: Human-perceived quality (1–4.5).
- **STOI:** Intelligibility score (0–1, higher is better).
- **SNR:** Signal clarity vs noise (in dB).

RMSE: 0.13819796
Spectrogram MSE: 300.34464
PESQ Score: 2.030520439147949
STOI Score: 0.607410905579882 3
SNR (dB): -3.4612494

Progress - Familiarity with Python Packages

Functionalities of Python we learned about through this project

os	File system access and path handling
librosa	Audio loading and feature extraction
soundfile	Reading/writing sound files (WAV)
pydub	Audio slicing and manipulation
subprocess	Run external programs like ffmpeg
pandas	Handle Excel files and dataframes
random	Add randomness to augmentations
numpy	Fast array operations, math
matplotlib.pyplot	Plotting for spectrogram visualization
librosa.display	Display waveform and spectrogram plots
shutil	High-level file operations (copy, move)
tqdm	Display progress bars in loops
torch	Deep Learning
torch.nn	Neural network layers and loss functions
torch.utils.data	DataLoader and dataset handling
torchvision.transforms	Image preprocessing and data augmentation
PIL.Image	Image loading, conversion, and resizing
sklearn.model_selection	Train/test data splitting
sklearn.metrics	Evaluate model performance



Progress - Research and Experimental Data

DATASET 1 - KAGGLE VEHICLE SOUNDS

- We used a subset of the Kaggle Vehicle Sounds Dataset, originally containing 9,448 labeled audio clips across 8 vehicle classes.
- For our project, we selected only the 4 most relevant classes: car(230), truck(265), motorcycle(537) and bus(4,221).
- These vehicle types are most commonly found in urban traffic, aligning closely with our surveillance use case at Plaksha.
- The selected subset presents a significant class imbalance (bus > car) which we have accounted for during preprocessing and model training using augmentation.

Progress - Research and Experimental Data

DATASET 2, 3 and final - Using data collected by the IoT Lab

DATASET 2

- To every 3 second span, allot 1 label with the highest confidence score.
- This confidence score was provided by the CV model that created the labels in the IoT lab.

DATASET 3

- Instead of a unit being a 3 second span, a unit is each label based on the csvs.
- For every label, create an audio clip starting one second before and ending one second after.

Issues - These were both single label, and mostly there was more than one type of vehicle passing by.

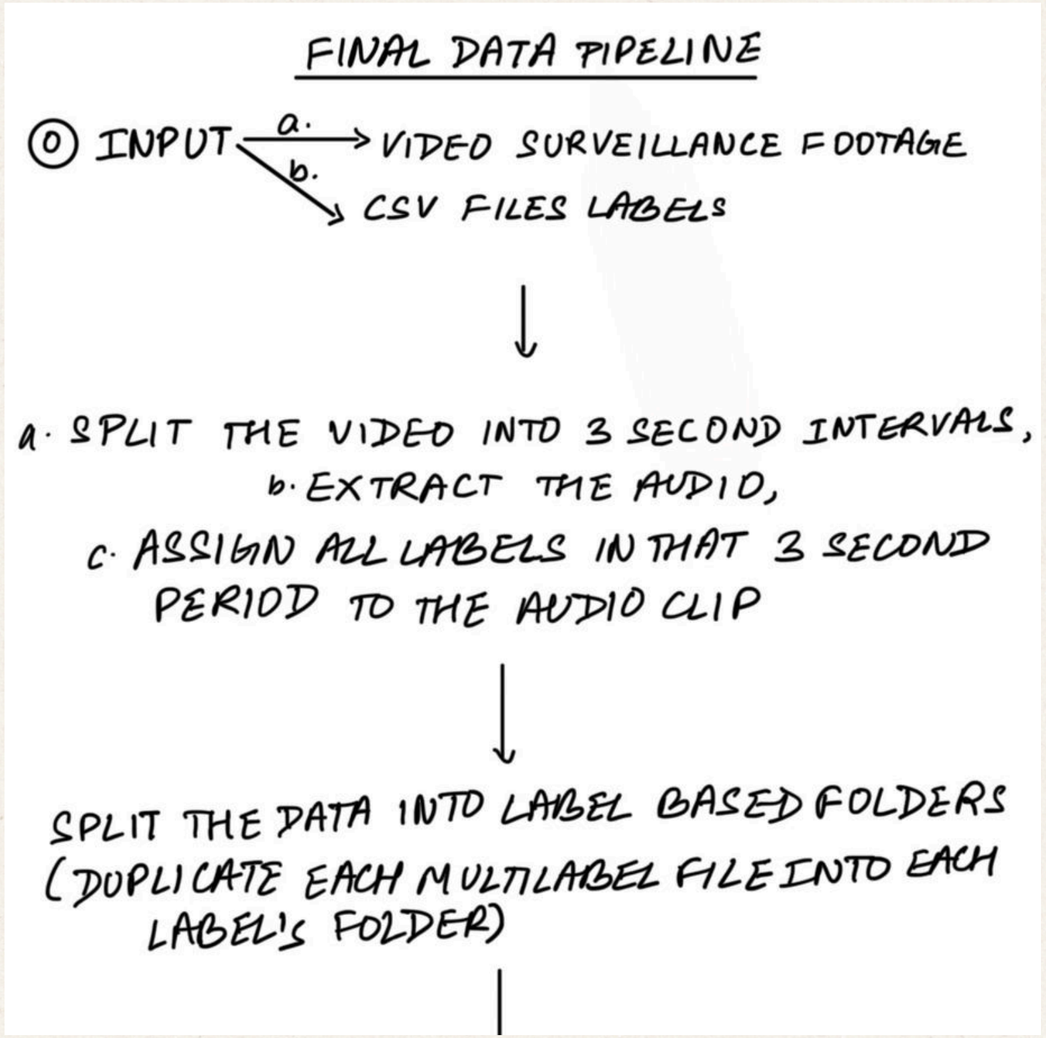
FINAL DATASET - CSV TO THE RIGHT

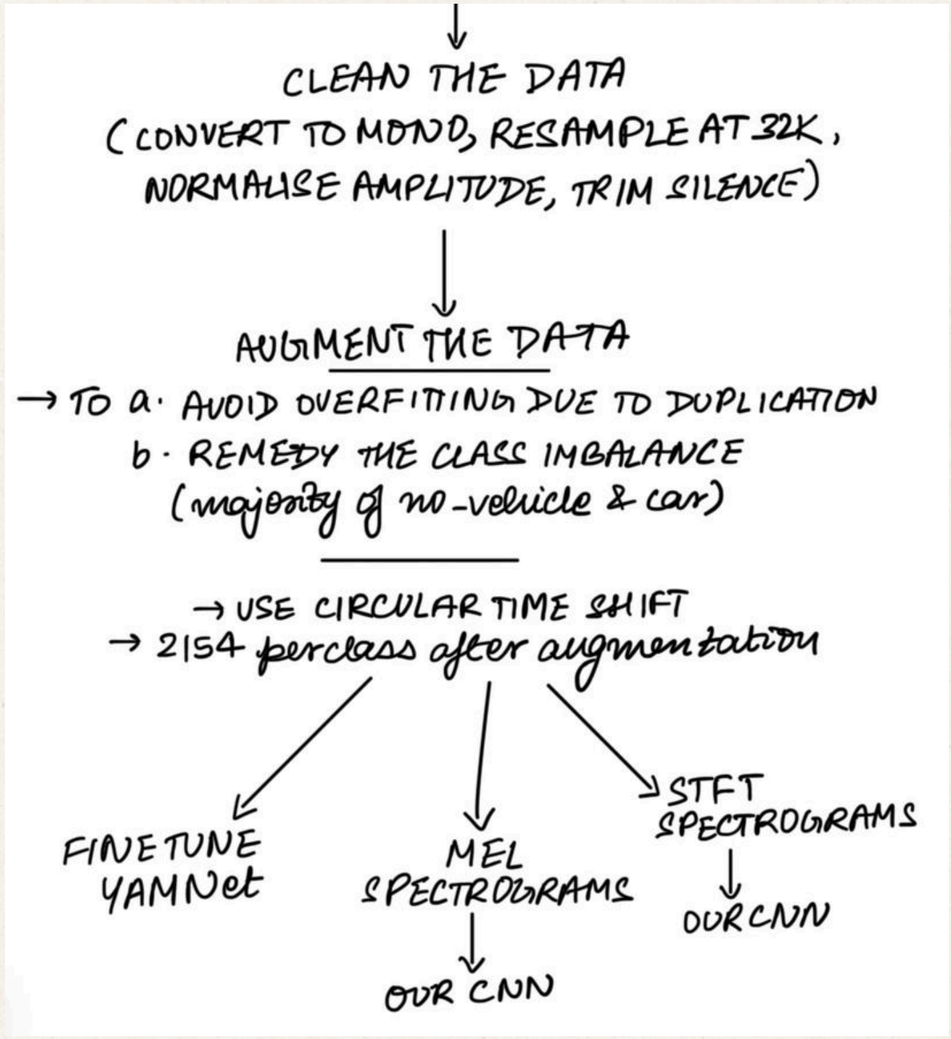
Per every 3 second clip, assign all labels present in the csv. The labels are car, truck, motorcycle and auto.

clip_start	clip_end	labels
0	3	['car', 'license-plate']
3	6	['car', 'license-plate', 'motorcycle', 'person', 'truck']
6	9	['autorickshaw', 'car', 'motorcycle', 'person']
9	12	['car']
12	15	['no_vehicle']
15	18	['no_vehicle']
18	21	['no_vehicle']
21	24	['no_vehicle']
24	27	['no_vehicle']
27	30	['car', 'license-plate']
30	33	['car', 'license-plate']
33	36	['car', 'license-plate']
36	39	['no_vehicle']
39	42	['no_vehicle']
42	45	['no_vehicle']
45	48	['no_vehicle']
48	51	['no_vehicle']
51	54	['no_vehicle']
54	57	['no_vehicle']
57	60	['no_vehicle']
60	63	['motorcycle', 'person']
63	66	['car', 'motorcycle', 'person']
66	69	['no_vehicle']
69	72	['car']
72	75	['car', 'license-plate']
75	78	['car', 'license-plate', 'motorcycle', 'person']
78	81	['no_vehicle']
81	84	['no_vehicle']
84	87	['car', 'motorcycle']
87	90	['car']
90	93	['no_vehicle']
93	96	['no_vehicle']
96	99	['no_vehicle']
99	102	['no_vehicle']
102	105	['no_vehicle']

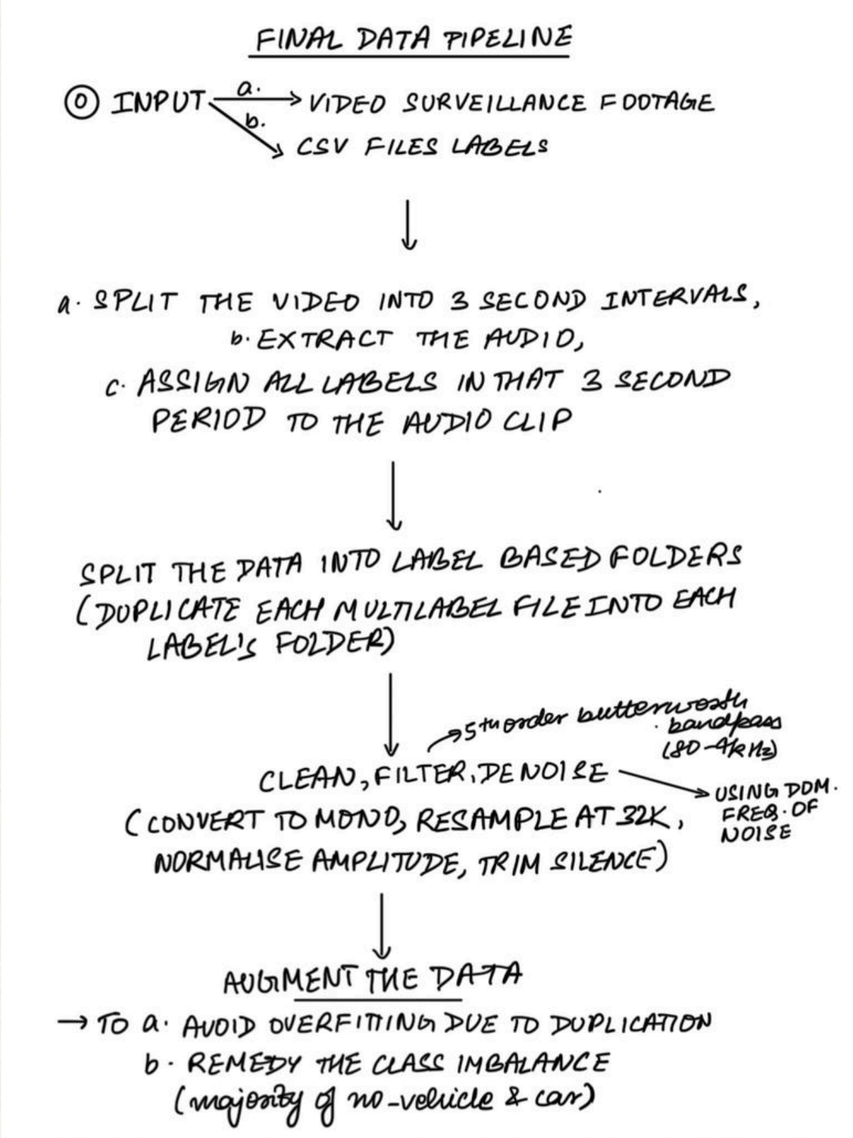
A csv used to create the final dataset of audios.

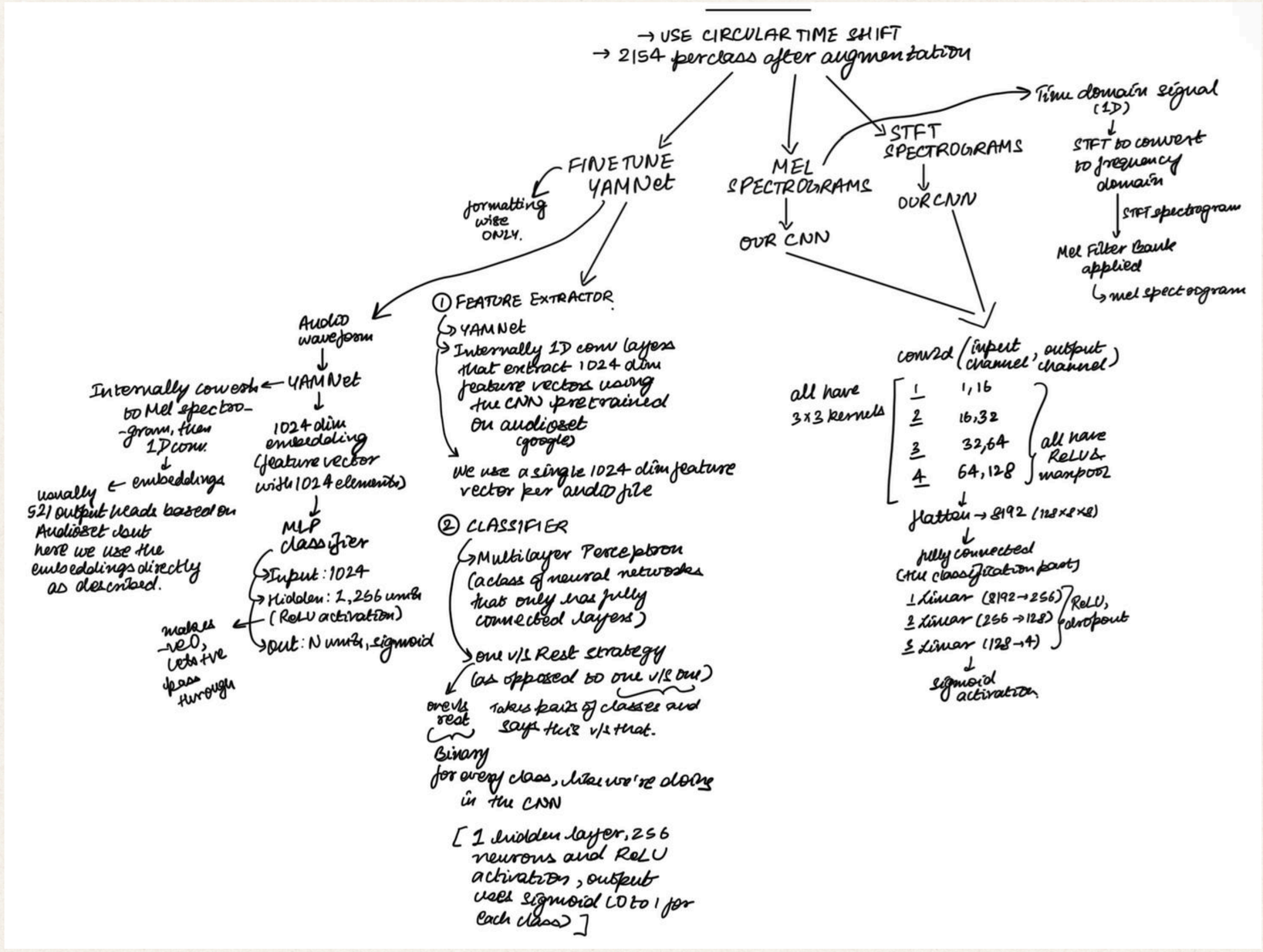
Our Final Pipeline (Summary)



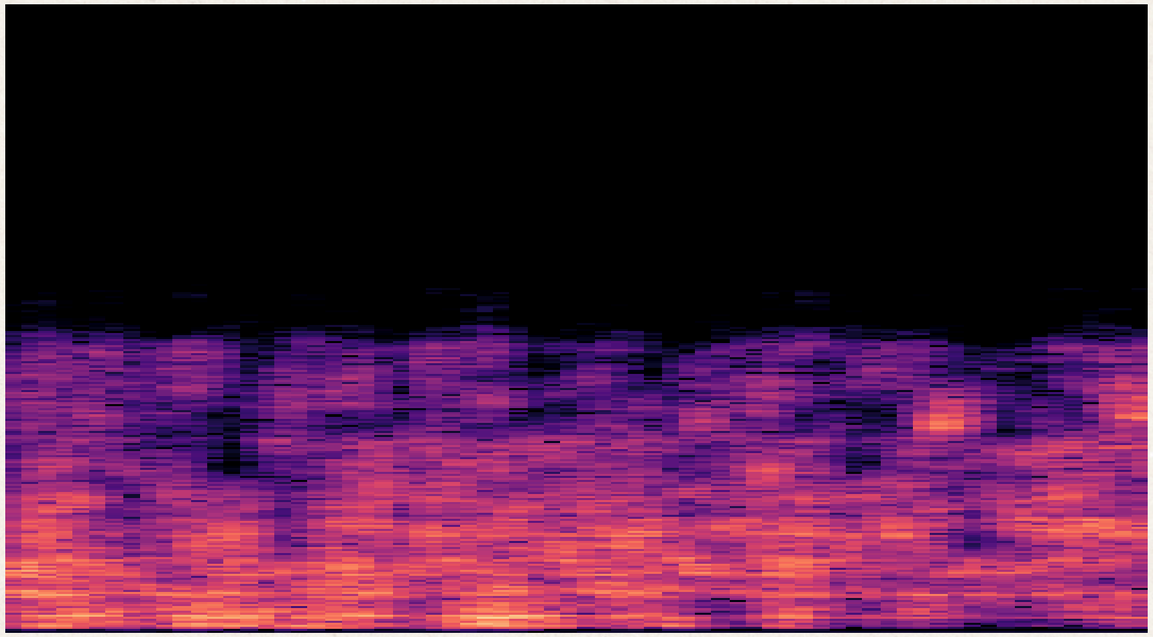


Our Final Pipeline (Detailed)

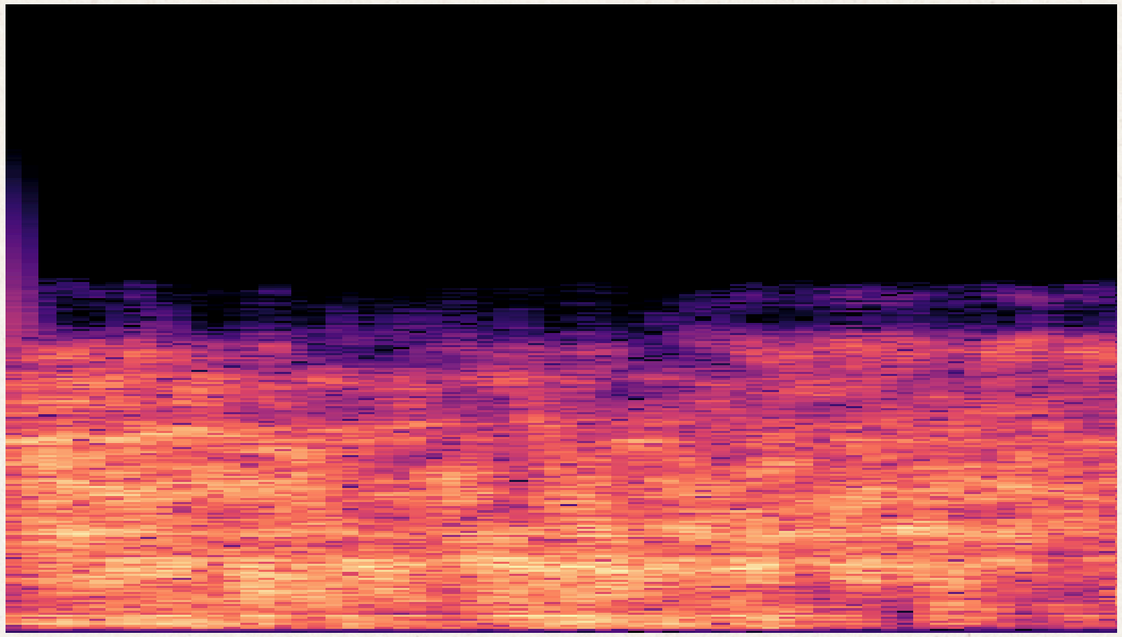




STFT Spectrograms - Theory



STFT Spectrogram for a motorcycle. (less loud, inconsistent)



STFT Spectrogram for a truck. (louder, more consistent)

STFT spectrograms show how frequency changes over time by taking overlapping short-time windows of the signal and computing the FFT for each window.

The result, as above, is a 2D representation where x-axis = time, y-axis = frequency, and color/intensity = magnitude.

Time-frequency tradeoff: Used large window size (2048) to prioritize frequency resolution over time resolution for clear distinction between vehicle engine frequencies and patterns rather than precise timing of events - optimal for vehicle type classification where acoustic signatures matters a little more than fast event detection.

Kaggle Dataset

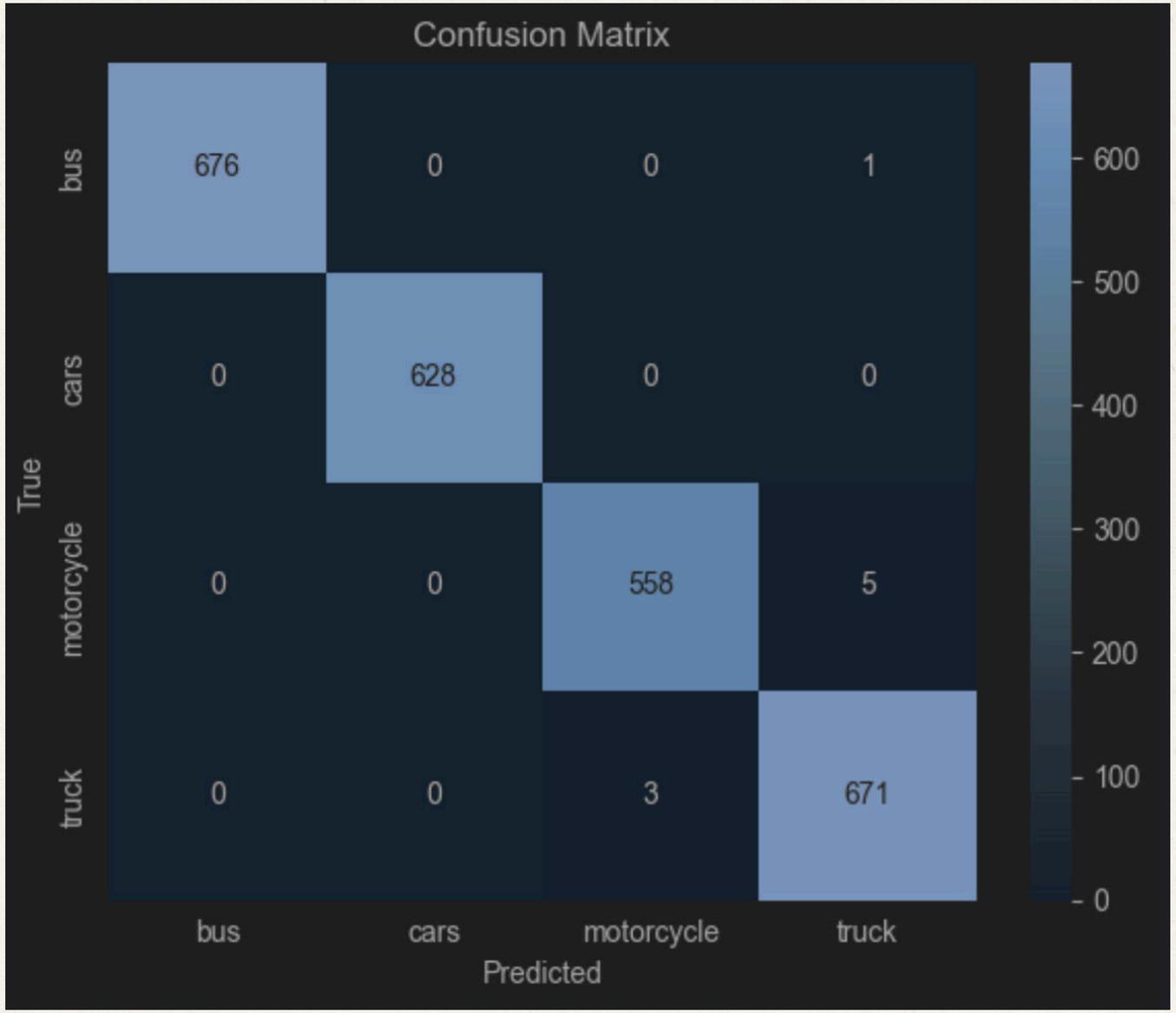
STFT Spectrograms - Our CNN

```
Classification Report:
              precision    recall  f1-score   support

         bus       1.00      1.00      1.00       677
        cars       1.00      1.00      1.00       628
  motorcycle       0.99      0.99      0.99       563
       truck       0.99      1.00      0.99       674

    accuracy                           1.00      2542
```

- Ensured no data leakage
- Dropout to prevent overfitting



Dataset from IoT Lab

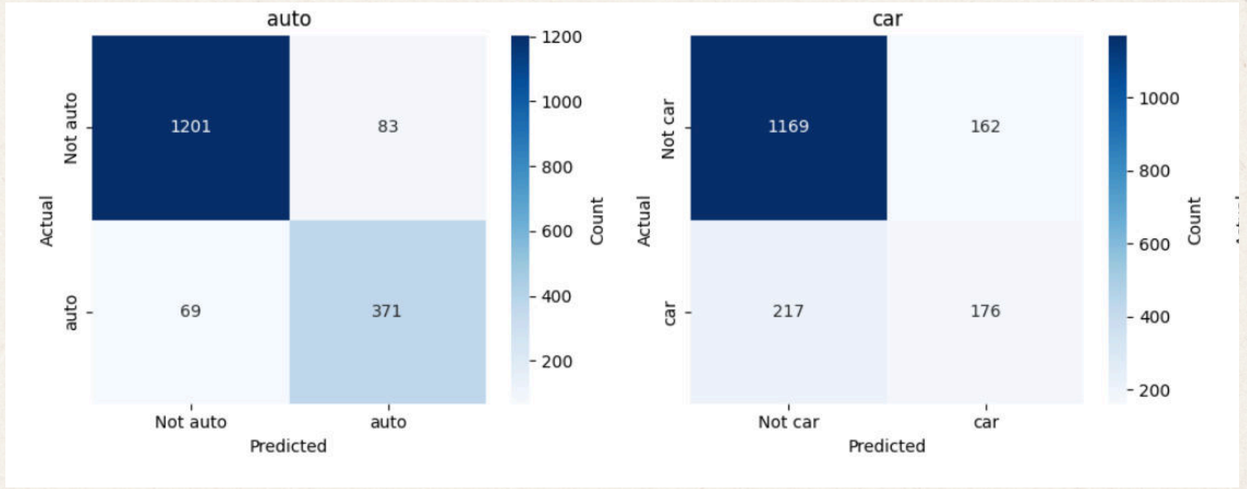
STFT Spectrograms - Our CNN

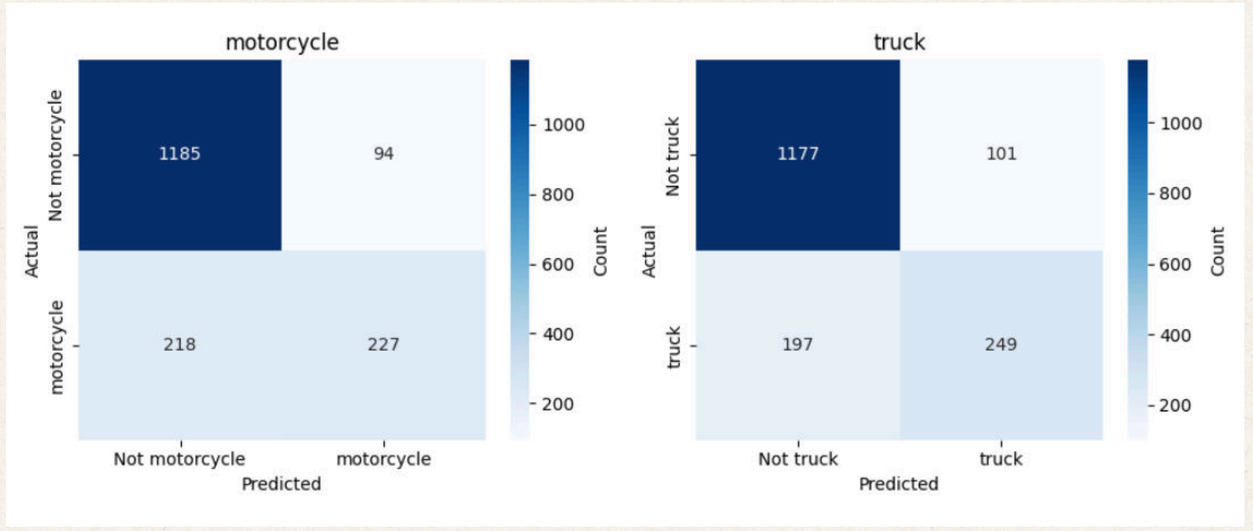
	precision	recall	f1-score	support
auto	0.82	0.84	0.83	440
car	0.52	0.45	0.48	393
motorcycle	0.71	0.51	0.59	445
truck	0.71	0.56	0.63	446
micro avg	0.70	0.59	0.64	1724
macro avg	0.69	0.59	0.63	1724
weighted avg	0.69	0.59	0.64	1724
samples avg	0.59	0.59	0.59	1724

```
Exact Match Accuracy: 0.5899
F1 Score (Micro): 0.6420
F1 Score (Macro): 0.6325
F1 Score (Weighted): 0.6364

Per-class Statistics:
auto       - Precision: 0.817, Recall: 0.843, F1: 0.830
car        - Precision: 0.521, Recall: 0.448, F1: 0.482
motorcycle - Precision: 0.707, Recall: 0.510, F1: 0.593
truck      - Precision: 0.711, Recall: 0.558, F1: 0.626

Plotting multilabel confusion matrix...
Saved confusion matrix to cnn_model_confusion_matrix.png
```





YAMnet

Libraries & Configuration

Load essential packages and set constants

Train-Test Split

Split data into 80% training and 20% testing sets

Model Training

Train OneVsRestClassifier with MLPClassifier

Save Model & Labels

Save trained classifier and label encoder



Load & Label Dataset

Iterate over directories and create a deduplicated dataset

Feature Extraction

Load YAMNet model and extract embeddings

Threshold

Setting up the threshold

Final Evaluation

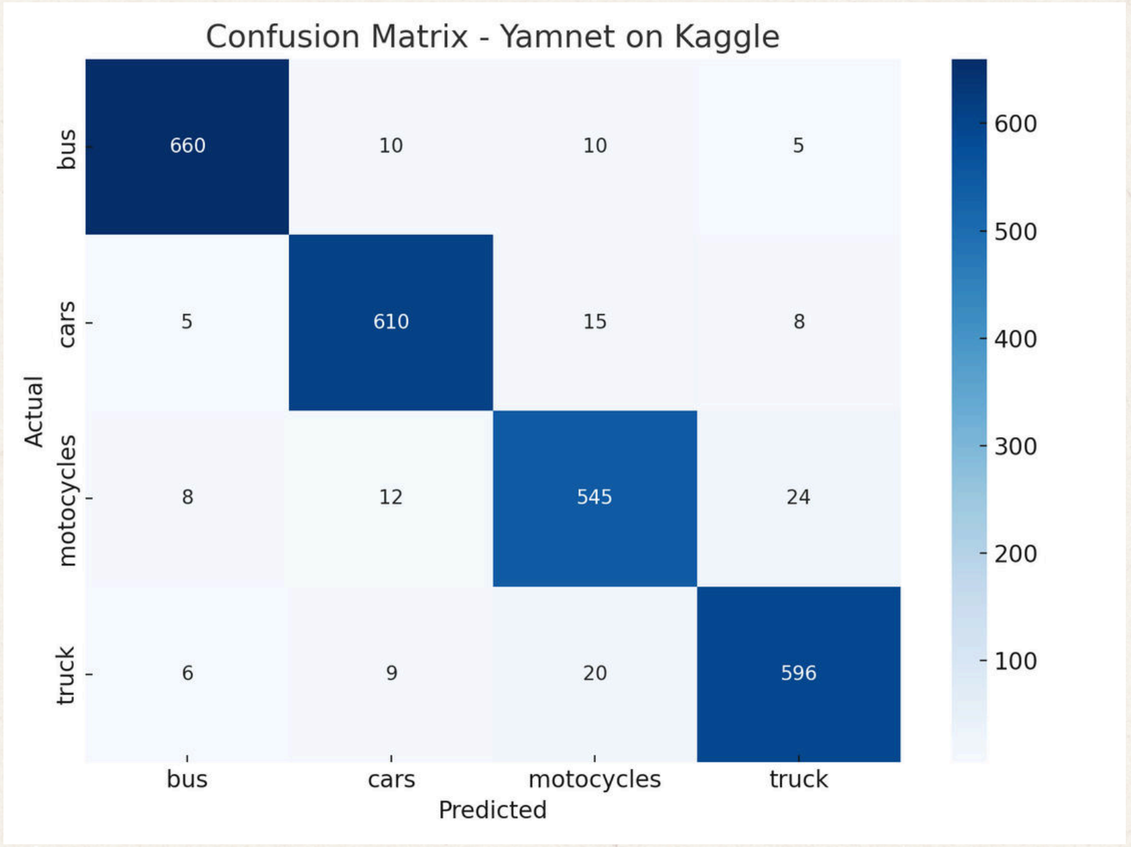
The labels that pass the threshold.

Dataset from Kaggle

YAMnet-Fine Tuned

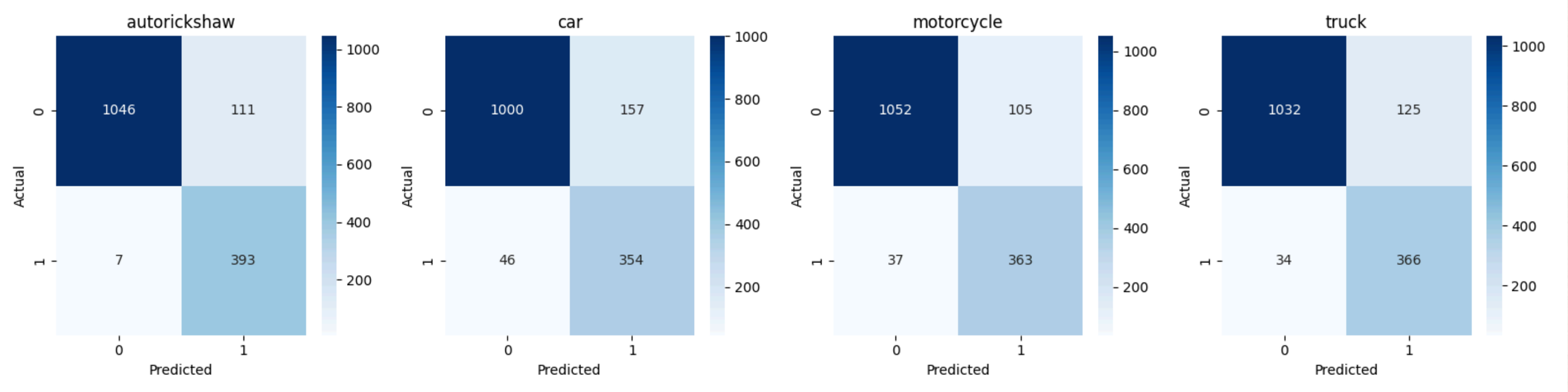
	Precision	Recall	F1-Score
bus	0.92	0.90	0.91
cars	0.93	0.93	0.93
motorcycles	0.91	0.93	0.92
truck	0.92	0.91	0.91

Test Accuracy: 0.9272



Dataset from IoT Lab

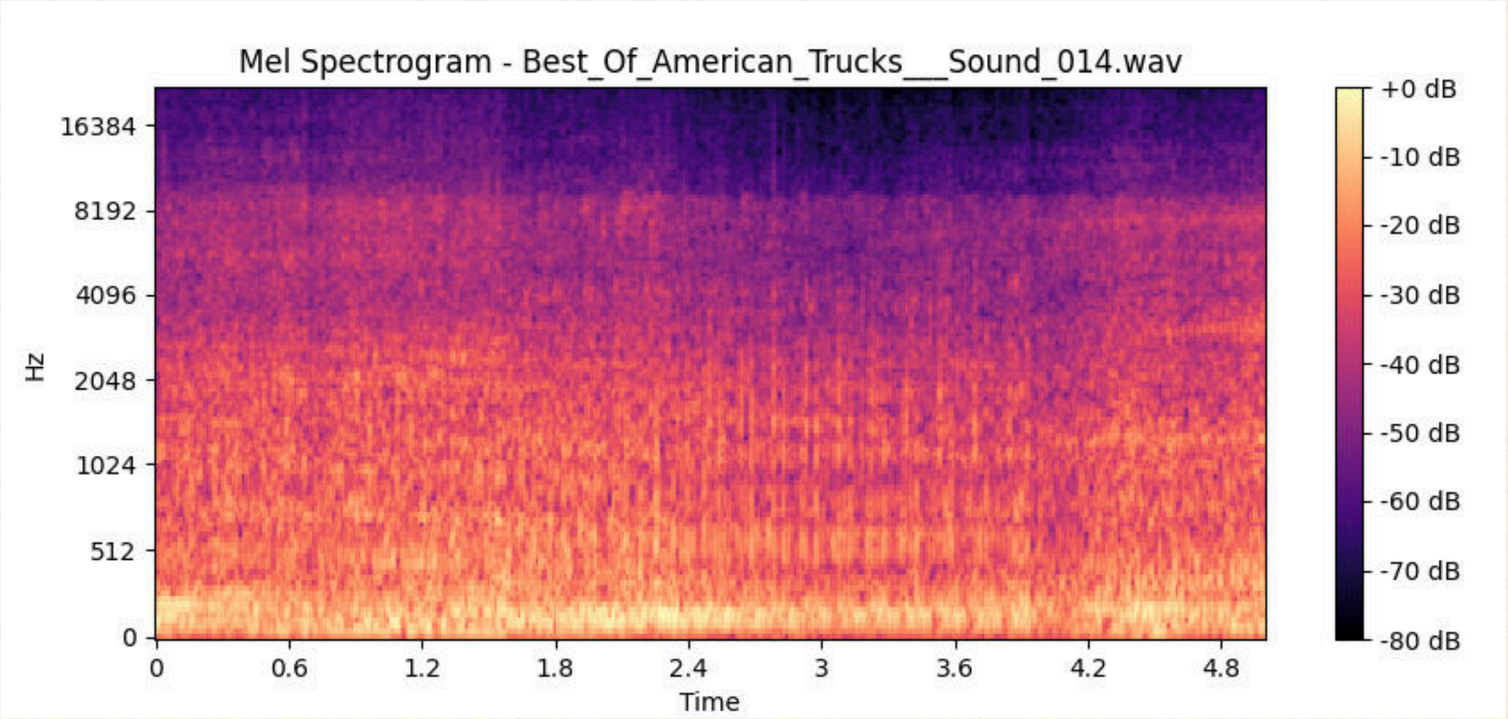
YAMnet-Fine Tuned



Class	Precision	Recall	F1-Score	Support
Autorickshaw	0.78	0.98	0.87	400
Car	0.69	0.89	0.78	400
Motorcycle	0.78	0.91	0.84	400
Truck	0.75	0.92	0.82	400

Exact Match Accuracy: 0.7270

A Mel-Spectrogram



(yellow for loud, purple/black for quiet), Bright horizontal below 512 Hz indicate strong low-frequency components typical of truck engine sounds, while the gradual fade of intensity at higher frequencies suggests engine noise and road friction.

To extract meaningful features from raw vehicle audio clips, we converted each .wav file into a Mel Spectrogram, which visually represents the sound's frequency content over time.

Dataset from IoT Lab

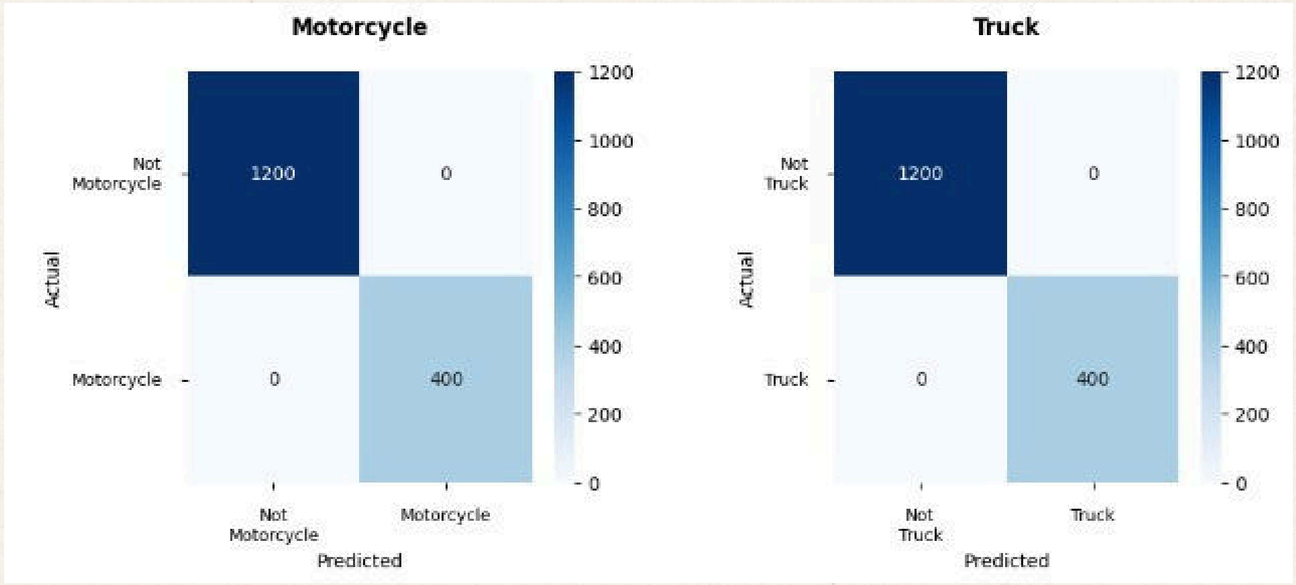
Mel Spectrograms - Our CNN

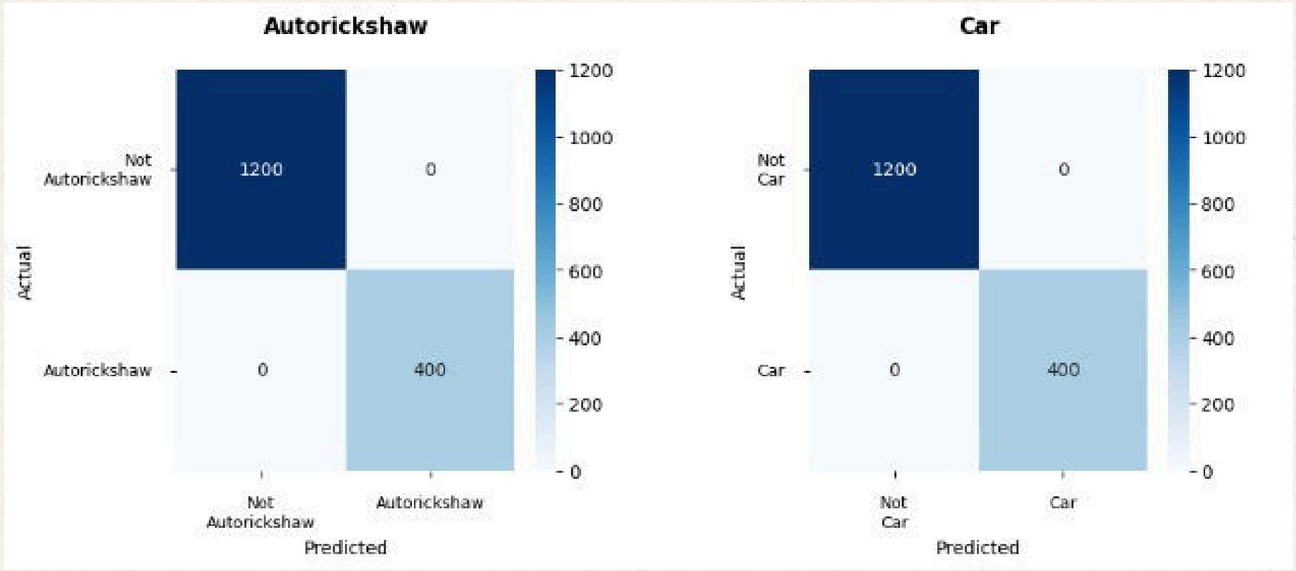
```
Classification Report (with threshold = 0.5):
                          precision    recall  f1-score   support

autorickshaw_spectrogram       1.00      1.00      1.00       447
        car_spectrogram        1.00      1.00      1.00       431
  motorcycle_spectrogram       1.00      1.00      1.00       429
      truck_spectrogram        1.00      1.00      1.00       417

              micro avg        1.00      1.00      1.00      1724
              macro avg        1.00      1.00      1.00      1724
           weighted avg        1.00      1.00      1.00      1724
            samples avg        1.00      1.00      1.00      1724

AUC for autorickshaw_spectrogram: 1.0000
AUC for car_spectrogram: 1.0000
AUC for motorcycle_spectrogram: 1.0000
AUC for truck_spectrogram: 1.0000
Average AUC: 1.0000
```



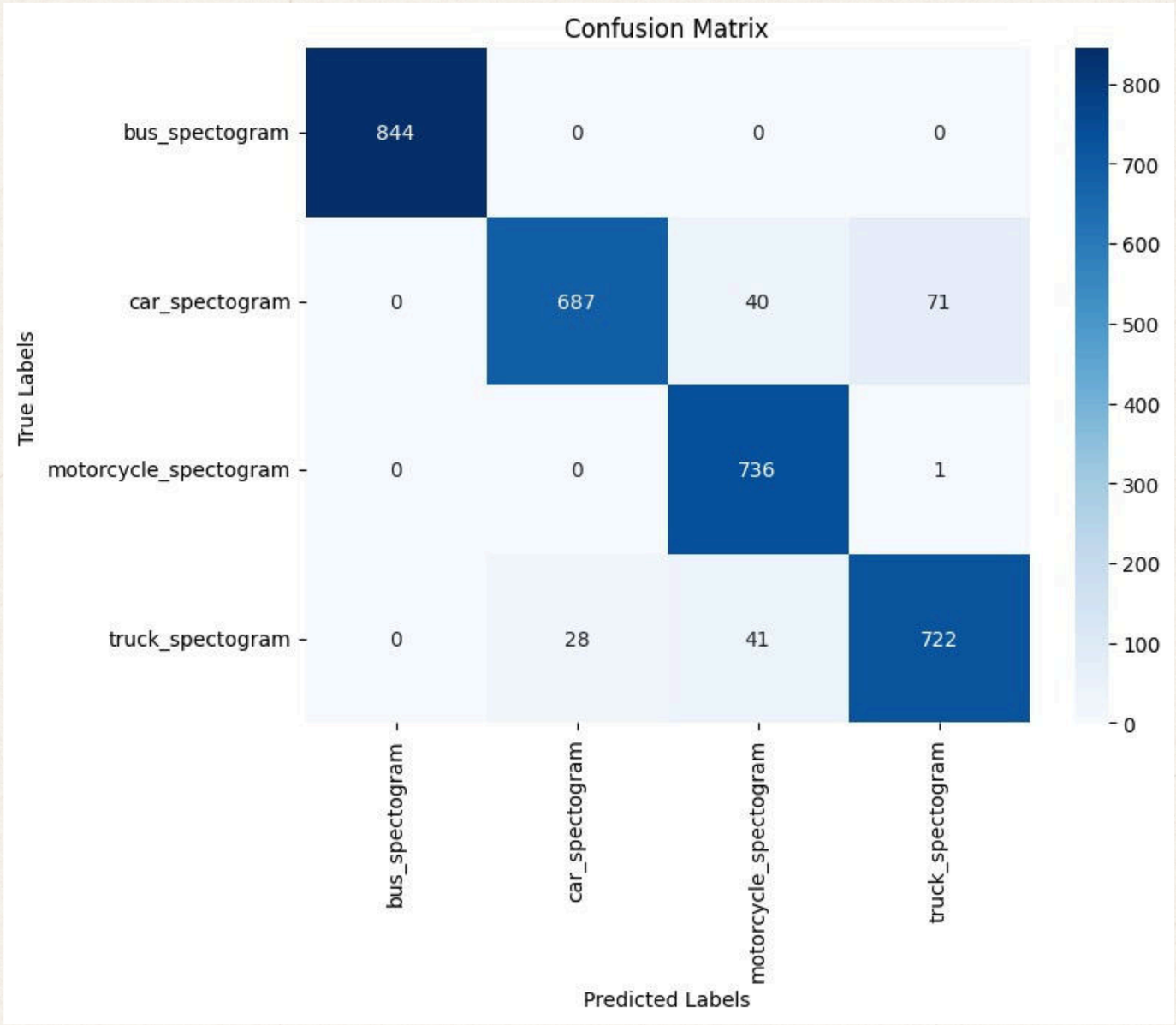


Kaggle Dataset

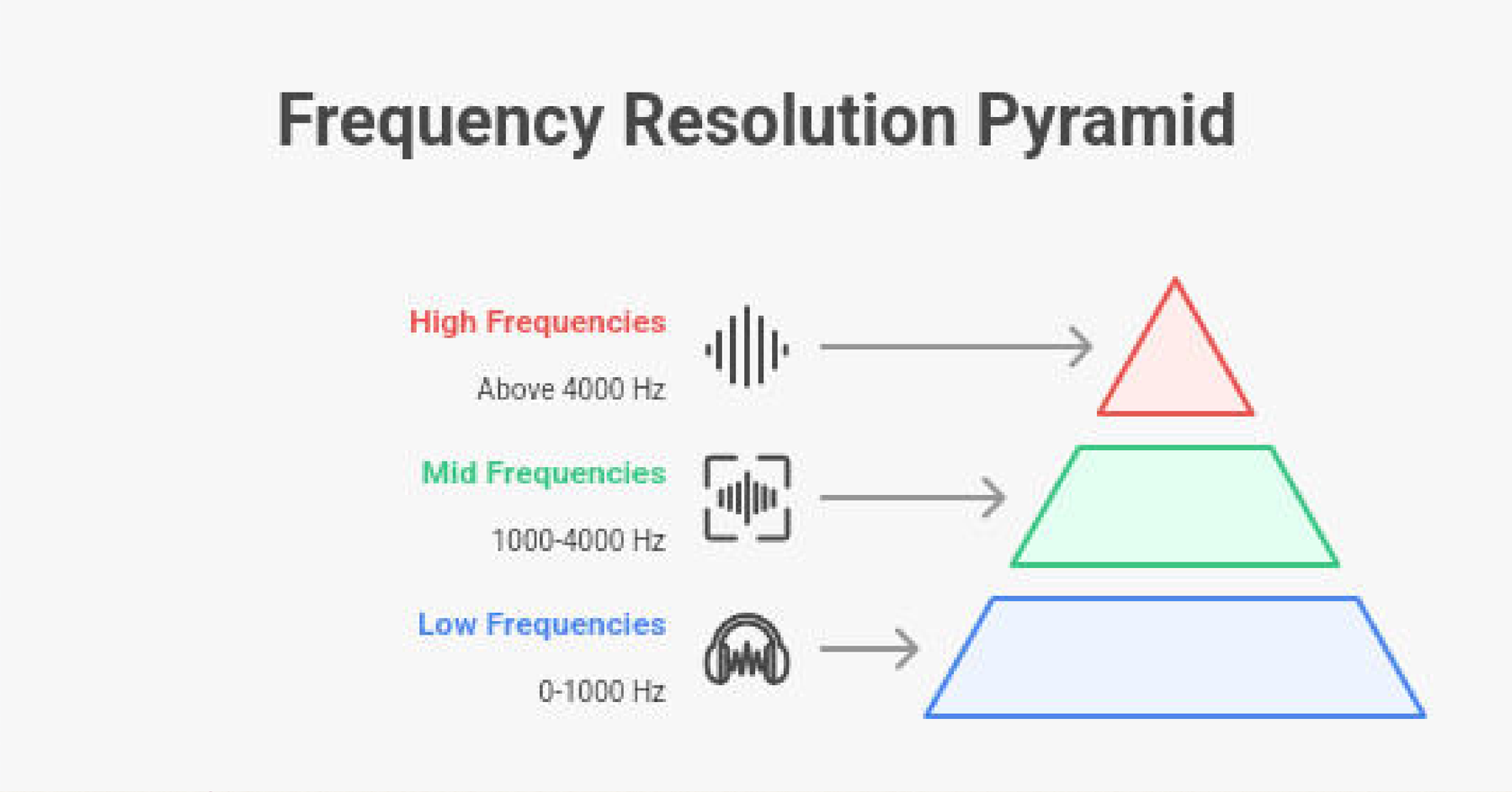
Mel Spectrograms - Our CNN

```
Test Loss: 0.1849
Test Accuracy: 0.9429
Model trained and saved successfully!
```

	precision	recall	f1-score
bus_spectogram	1.00	1.00	1.00
car_spectogram	0.96	0.86	0.91
motorcycle_spectogram	0.90	1.00	0.95
truck_spectogram	0.91	0.91	0.91
accuracy			0.94
macro avg	0.94	0.94	0.94
weighted avg	0.94	0.94	0.94



Why we think Mel Performs the Best



AUDIO PROCESSING LITERATURE REVIEW

1. Mel Spectrogram-Based Approaches

- Malakar et al. (2024) utilized Mel spectrograms in conjunction with YOLOv8 for deepfake audio detection, showing that image-based object detection techniques can be successfully adapted for spectrogram analysis [Malakar et al., 2024].
- Adithya & Pradeep (2022) explored vehicle detection and speed estimation using acoustic signals transformed into Mel spectrograms, achieving robust classification and estimation performance [Adithya & Pradeep, 2022].

2. Non-Mel Spectrogram and Learnable Representations

- Tian et al. (2022) proposed SpectNet, which integrates spectrogram generation as a learnable module within the network. This enabled end-to-end optimization and improved performance over traditional feature extraction methods [Tian et al., 2022].
- Pampalk et al. (2008) demonstrated that treating spectrograms as texture images allows for effective classification of musical instruments using visual features, showcasing the versatility of spectrogram-based methods [Pampalk et al., 2008].

AUDIO PROCESSING LITERATURE REVIEW

3. FFT-Based Audio Representations

- Kumar & Srivastava (2016) applied FFT in a real-time audio-visual speaker tracking system using a Kalman filter. FFT enabled fast frequency domain analysis essential for tracking in noisy environments [Kumar & Srivastava, 2016].
- Puckette (2000) demonstrated FFT-based real-time audio processing for music applications, highlighting the low-latency advantages and interpretability of FFT features [Puckette, 2000].

4. YAMNet, a pre-trained deep neural network based on MobileNetV1 and trained on AudioSet, has become a powerful backbone for audio classification tasks.

- Zafar et al. (2024) showed that fine-tuning YAMNet significantly enhanced gunshot detection in urban environments, using its embeddings as features for downstream classifiers [Zafar et al., 2024].
- Lee et al. (2024) proposed W-YAMNet, an architecture optimized for water-usage activity detection in smart homes. They demonstrated the utility of YAMNet embeddings in classifying complex acoustic patterns [Lee et al., 2024].
- The TensorFlow Audio Transfer Learning tutorial provides a practical example of reusing YAMNet embeddings for custom environmental sound classification, making it accessible for prototyping and research [TensorFlow, 2022].

Future Plans

- **Multimodal Integration:** Combine acoustic data with visual or radar-based methods for improved accuracy.

- **Real-Time Implementation:** Develop a real-time vehicle detection system for practical deployment on roads and traffic based areas

- **Creating our own dataset:** For working on the specific - video survalence and traffic based location.

- **Using an ensemble of models for more accurate audio classification.**

- Use a **contrastive learning** pre-training framework where:
  - Positive pair: Two audio clips from similar vehicle types.
  - Negative pair: Audio clips from different vehicle types.
  - The CNN will learn to group similar vehicle sounds closer in feature space.

Thank you

Metrics Used

$$\text{F1 Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

